BLUEPRINT - WEBSITE BUILDER PHASE 2

Next Generation E-commerce Platform

📋 RESUMEN EJECUTIVO

Fecha: 11 Agosto 2025
Fase: 2 - Componentes Estructurales y Páginas
Stack: ASP.NET Core 8 + PostgreSQL | Next.js 14 + TypeScript
Estrategia: JSONB modular, archivos máx 300 líneas

🎯 OBJETIVOS FASE 2

1. Implementar 4 componentes globales (Header, Announcement Bar, Footer, Cart Drawer)
2. Definir 8 tipos de páginas del website builder
3. Sistema drag & drop con validaciones jerárquicas
4. Sistema undo/redo con 50 estados + atajos teclado
5. Mantener arquitectura modular para fácil mantenimiento

🏗️ ARQUITECTURA DE BASE DE DATOS

Tablas Principales

```
-- Configuraciones globales del tema (YA EXISTE - Fase 1)
ThemeSettings {
  Id: int (PK)
  StoreId: int (FK)
  SettingType: string -- 'global', 'header', 'footer', 'announcement', 'cart'
  Settings: JSONB
  IsActive: bool
  CreatedAt: DateTime
  UpdatedAt: DateTime
```

```
}

-- Páginas del website
WebsitePages {
  Id: int (PK)
  StoreId: int (FK)
  PageType: string -- 'home', 'product', 'cart', 'checkout', 'custom', 'all-collections', 'collection',
'all-products'
  Slug: string (unique)
  Title: string
  MetaData: JSONB
  Sections: JSONB -- Array de secciones (Fase 3)
  IsPublished: bool
  PublishedAt: DateTime?
  CreatedAt: DateTime
  UpdatedAt: DateTime
}

-- Historial para undo/redo
EditorHistory {
  Id: int (PK)
  StoreId: int (FK)
  EntityType: string -- 'theme', 'page', 'section'
  EntityId: int
  State: JSONB
  Action: string
  UserId: int
  CreatedAt: DateTime
}
```

🔧 COMPONENTES GLOBALES (1 instancia por tienda)

1. HEADER

Ubicación DB: ThemeSettings (SettingType: 'header')

```
interface HeaderSettings {
  // General
  colorScheme: 'default' | 'scheme1' | 'scheme2' | 'scheme3' | 'scheme4' | 'scheme5'
  width: 'screen' | 'page' | 'large' | 'medium'
  layout: 'drawer' | 'logo-left-menu-center' | 'logo-left-menu-left' |
          'logo-center-menu-left' | 'logo-center-menu-below' | 'logo-left-menu-left-below'
```

```
  showDivider: boolean
  headerHeight: number // 0-200px
  enableStickyHeader: boolean

  // Menu
  menuOpenOn: 'click' | 'hover'
  navigationMenuId: string // FK to navigation module

  // Logo
  desktopLogo: {
    url: string
    alt: string
  }
  desktopLogoSize: number // px
  mobileLogo: {
    url: string
    alt: string
  }
  mobileLogoSize: number // px

  // Icons
  showSearchIcon: boolean
  showAccountIcon: boolean
  iconStyle: 'solid' | 'outline'
  cartIconType: 'cart' | 'bag'
}
```

2. ANNOUNCEMENT BAR

Ubicación DB: ThemeSettings (SettingType: 'announcement')

```
interface AnnouncementBarSettings {
  // Parent settings
  showOnlyHomePage: boolean
  colorScheme: 'default' | 'scheme1' | 'scheme2' | 'scheme3' | 'scheme4' | 'scheme5'
  width: 'screen' | 'page' | 'large' | 'medium'
  showNavigationArrows: boolean
  autoplay: {
    enabled: boolean
    mode: 'none' | 'one-at-time'
    speed: number // seconds
  }
  animationStyle: 'slide-horizontal' | 'none' | 'fade' |
```

```
                'slide-vertical' | 'typewriter' | 'marquee'
  showLanguageSelector: boolean
  showCurrencySelector: boolean
  showSocialIcons: boolean

  // Child announcements (multi-instance)
  announcements: Array<{
    id: string
    text: string // Rich text HTML
    visible: boolean
    link?: string
    useIcon: boolean
    icon?: string
  }>
}
```

3. FOOTER

Ubicación DB: ThemeSettings (SettingType: 'footer')

```
interface FooterSettings {
  // General
  colorScheme: 'default' | 'scheme1' | 'scheme2' | 'scheme3' | 'scheme4' | 'scheme5'
  hasBackgroundColor: boolean
  width: 'container' | 'screen'
  desktopColumns: 3 | 4
  showSeparator: boolean
  footerHeight: number // px

  // Bottom bar
  showBottomBar: boolean
  bottomBarMenuId?: string // FK to navigation module
  showPaymentIcons: boolean
  copyrightText: string

  // Blocks (multi-instance, ordenables)
  blocks: Array<FooterBlock>
}

// Footer block types
type FooterBlock =
  | LogoWithTextBlock
  | SubscribeBlock
```

```
  | SocialMediaBlock
  | MenuBlock
  | TextBlock

interface LogoWithTextBlock {
  type: 'logo-with-text'
  id: string
  logo: {
    url: string
    alt: string
  }
  logoSize: number
  heading: string
  body: string // Rich text HTML
}

interface SubscribeBlock {
  type: 'subscribe'
  id: string
  heading: string
  body: string // Rich text HTML
  inputStyle: string
}

interface SocialMediaBlock {
  type: 'social-media'
  id: string
  heading: string
  iconStyle: 'monochrome' | 'color'
  iconSize: 'small' | 'medium' | 'large'
  iconSpacing: number
  networks: {
    facebook?: { enabled: boolean; url: string }
    instagram?: { enabled: boolean; url: string }
    twitter?: { enabled: boolean; url: string }
    pinterest?: { enabled: boolean; url: string }
    youtube?: { enabled: boolean; url: string }
    tiktok?: { enabled: boolean; url: string }
    linkedin?: { enabled: boolean; url: string }
    snapchat?: { enabled: boolean; url: string }
    // ... más redes
  }
}
```

```
interface MenuBlock {
  type: 'menu'
  id: string
  heading: string
  menuId: string // FK to navigation module
}

interface TextBlock {
  type: 'text'
  id: string
  heading: string
  body: string // Rich text HTML
}
```

4. CART DRAWER

Ubicación DB: ThemeSettings (SettingType: 'cart')

```
interface CartDrawerSettings {
  // General
  colorScheme: 'default' | 'scheme1' | 'scheme2' | 'scheme3' | 'scheme4' | 'scheme5'
  imageRatio: 'default' | 'portrait' | 'square'
  showOrderNotes: boolean
  showTaxesAndShipping: boolean

  // Display
  displayAs: 'drawer' | 'page' | 'drawer-and-page'
  showProgressBar: boolean
  freeShippingGoal: number // Amount for free shipping

  // Progress bar
  progressBarGradient: string // CSS gradient
  darkenImageBackground: boolean

  // Button texts
  checkoutButtonText: string
  viewCartButtonText: string
  continueShoppingButtonText: string

  // Styling
  borderRadius: 'size-1' | 'size-2' | 'size-3' | 'none'
  width: 'extra-small' | 'small' | 'large' | 'full-width'
```

```
  // Spacing
  topSpacing: number // 0-120px
  bottomSpacing: number // 0-120px
  addLateralSpacing: boolean

  // Advanced
  paymentButtonText: string
  customCSS?: string
}
```

📄 TIPOS DE PÁGINAS (8 tipos)

Páginas del Website Builder

```
enum PageType {
  HOME = 'home',                    // Página principal (única)
  PRODUCT = 'product',              // Página de producto individual
  CART = 'cart',                    // Página del carrito
  CHECKOUT = 'checkout',            // Página de checkout
  CUSTOM = 'custom',                // Páginas personalizadas
  ALL_COLLECTIONS = 'all-collections', // Vista todas las colecciones
  COLLECTION = 'collection',        // Página de colección específica
  ALL_PRODUCTS = 'all-products'     // Catálogo todos los productos
}

interface PageMetadata {
  seoTitle?: string
  seoDescription?: string
  seoImage?: string
  customHandle?: string
  templateSettings?: Record<string, any>
}
```

🔄 SISTEMA DRAG & DROP

Validaciones Jerárquicas

```
interface DragDropRules {
  // Secciones permitidas por tipo de página
  allowedSections: {
```

```
    [PageType.HOME]: string[]
    [PageType.PRODUCT]: string[]
    [PageType.COLLECTION]: string[]
    // etc...
  }

  // Límites de instancias
  sectionLimits: {
    [sectionType: string]: number | 'unlimited'
  }

  // Orden requerido
  requiredOrder: {
    first?: string[]  // Secciones que deben ir primero
    last?: string[]   // Secciones que deben ir último
  }
}
```

↩️ SISTEMA UNDO/REDO

Especificaciones

```
interface UndoRedoSystem {
  maxStates: 50
  shortcuts: {
    undo: 'Ctrl+Z' | 'Cmd+Z'
    redo: 'Ctrl+Y' | 'Cmd+Y' | 'Ctrl+Shift+Z' | 'Cmd+Shift+Z'
  }

  // Estados guardados
  stateTypes: [
    'component-update',    // Cambios en header/footer/etc
    'section-add',         // Agregar sección
    'section-remove',      // Eliminar sección
    'section-reorder',     // Reordenar secciones
    'setting-change',      // Cambio de configuración
    'content-edit'         // Edición de contenido
  ]

  // Debounce para agrupar cambios
  debounceMs: 1000
}
```

💾 ESTRATEGIA DE CACHE

```
// PRODUCCIÓN: Cache 24 horas
[ResponseCache(Duration = 86400, VaryByHeader = "Accept-Language")]

// PREVIEW: Cache 5 minutos
[ResponseCache(Duration = 300, Location = ResponseCacheLocation.Client)]

// Invalidación selectiva
- Al guardar Header → Invalidar cache 'header'
- Al guardar Footer → Invalidar cache 'footer'
- Al publicar página → Invalidar cache específico de página
```

🎨 VALORES POR DEFECTO (Jacaranda Plaza)

```
const DEFAULT_THEME = {
  header: {
    colorScheme: 'scheme5',
    width: 'large',
    layout: 'drawer',
    headerHeight: 160,
    desktopLogoSize: 116,
    mobileLogoSize: 50,
    iconStyle: 'outline',
    cartIconType: 'cart'
  },

  announcementBar: {
    colorScheme: 'scheme1',
    width: 'screen',
    animationStyle: 'slide-horizontal',
    autoplay: {
      enabled: false,
      mode: 'none',
      speed: 4
    }
  },

  footer: {
```

```
    colorScheme: 'scheme1',
    width: 'screen',
    desktopColumns: 3,
    footerHeight: 200
  },

  cartDrawer: {
    colorScheme: 'scheme2',
    imageRatio: 'portrait',
    displayAs: 'drawer-and-page',
    freeShippingGoal: 0,
    borderRadius: 'size-2',
    width: 'extra-small'
  }
}
```

📁 ESTRUCTURA DE ARCHIVOS

```
/backend
├── Models/
│   ├── Theme/
│   │   ├── HeaderSettings.cs
│   │   ├── AnnouncementBarSettings.cs
│   │   ├── FooterSettings.cs
│   │   └── CartDrawerSettings.cs
│   ├── Pages/
│   │   ├── WebsitePage.cs
│   │   └── PageMetadata.cs
│   └── Editor/
│       └── EditorHistory.cs
│
├── Controllers/
│   ├── ThemeController.cs
│   ├── PagesController.cs
│   └── EditorController.cs
│
└── Services/
    ├── ThemeService.cs
    ├── PageService.cs
    └── CacheService.cs

/frontend
```

```
├── types/
│   ├── theme/
│   │   ├── header.ts
│   │   ├── announcement-bar.ts
│   │   ├── footer.ts
│   │   ├── cart-drawer.ts
│   │   └── index.ts
│   ├── pages/
│   │   ├── page-types.ts
│   │   └── index.ts
│   └── editor/
│       ├── drag-drop.ts
│       └── undo-redo.ts
```

⚠️ CONSIDERACIONES TÉCNICAS

1. **Modularidad**: Ningún archivo debe exceder 300 líneas
2. **JSONB**: Usar para flexibilidad en Settings y Sections
3. **Validación**: Implementar en backend y frontend
4. **Performance**: Cache diferenciado preview/production
5. **Migraciones**: Incluir valores por defecto
6. **Testing**: Unit tests para validaciones críticas

🚀 PRÓXIMOS PASOS

Fase 2 - Inmediato:

1. ✅ Crear modelos C# para Entity Framework
2. ✅ Crear types TypeScript modulares
3. ✅ Implementar controladores básicos
4. ✅ Sistema drag & drop
5. ✅ Sistema undo/redo

Fase 3 - Siguiente:

- 11 tipos de secciones modulares
- Validaciones por tipo de página
- Sistema de templates

Fase 4 - Final:

- UI del editor completa
- Preview en tiempo real
- Publicación y versionado

📊 MÉTRICAS DE ÉXITO

- [ ] Archivos < 300 líneas
- [ ] Cache funcionando (24h prod / 5min preview)
- [ ] Drag & drop sin bugs
- [ ] Undo/redo 50 estados
- [ ] Sin JSON gigante (modular JSONB)
- [ ] Performance < 2s carga inicial

Documento preparado para Claude Code - Fase 2 Implementation